Homework-01 – After mid

Embedded IOT Systems Fall2025

CS(A-B) AI 5th

Question-1 ESP32 Webserver (webserver.cpp)

Part A: Short Questions

1. **What is the purpose of WebServer server(80); and what does port 80 represent?**
   WebServer server(80) creates the HTTP web server on ESP32 that receives the incoming client requests.
   Port 80 is default web browser port for HTTP communication. It is automatically used when there is no specific port. This allows user to access ESP32 webpage using its IP address through a browser.

2. **Explain the role of server.on("/", handleRoot); in this program.**
   This line tells ESP32 what to do when someone opens its webpage. '/' means the main webpage of the web server. When a user types the ESP32's IP address in a web browser, ESP32 understands that the user is asking for its main webpage. It then calls the handleRoot() function. In this function, ESP32 creates an HTML page and puts the latest temperature and humidity values into that page then page is sent back to the browser.

3. **Why is server.handleClient(); placed inside the loop() function? What will happen if it is removed?**
   This line tells the ESP32 to keep checking if some client is trying to open its webpage. The loop allows it to run again and again and checks for browser requests all the time. If this line is not there, ESP32 will not check for browser requests and will not answer them and webpage will stop working.

4. **In handleRoot(), explain the statement: server.send(200, "text/html", html);**
   This statement is used to send webpage from ESP32 to the browser. 200 means everything is ok. "text/html" tells the data that is being sent is a webpage.
   Webpage is sent and loaded successfully.

5. **What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside handleRoot()?**

Displaying last measured sensor values	**Taking fresh readings using handleRoot()**
Sensor will read only when the button is pressed	Sensor reads everytime the browser page is refreshed or opened
Webpage loads fast and smoothly	Webpage loads slowly
Gives stable readings	May give unstable readings

Part B: Long Question

Describe the complete working of the ESP32 webserver-based temperature and humidity monitoring system.

Your answer should include:

- ESP32 Wi-Fi connection process and IP address assignment

First ESP32 connects to a Wi-Fi network using WiFi.begin(ssid, password). It tries to connect the given wifi and when connection is successful it displays the ESP32 ip address on serial monitor which can be used to open EESP32 webpage on a browser.

- Web server initialization and request handling

A webserver is created on port 80 and the main webpage is linked to the handleRoot() function which will sends the ESP32 webpage on browser. The server is started and handleClient() keeps checking for the client's requests and responds them.

- Button-based sensor reading and OLED update mechanism

A pushbutton uses input pullup(high) connected to ESP32 on a GPIO pin. Everytime a button is pressed, sensor will read temperature and humidity which is displayed on oled. Oled readings will update everytime the new readings are taken.

- Dynamic HTML webpage generation

When a browser open ESP32 webpage, handleRoot() function runs, HTML webpage is created and readings are send to the browser on to the webpage. The webpage updates automatically with new readings when button is pressed.

- Purpose of meta refresh in the webpage

The line "<meta http-equiv='refresh' content='5'>"; automatically refreshes the webpage every 5 seconds and shows updated values. This allows webpage to become a live monitor.

- Common issues in ESP32 webserver projects and their solutions

- Wifi not connecting then should check the ssid and password and also whether it is connected to the device on which browser is being opened.
- Webpage not opening then should verify the ESP32 IP address
- Wrong sensor values then should check correct DHT sensor type.

Question-2 Blynk Cloud Interfacing (blynk.cpp)

Part-A: Short Questions

1. **What is the role of Blynk Template ID in an ESP32 IoT project? Why must it match the cloud template?**
   Template ID tells the ESP32 which Blynk project it should connect to on the cloud and it must match the cloud template so that it can send the data properly to that cloud for which it is specified as it links the device to correct dashboard.

2. **Differentiate between Blynk Template ID and Blynk Auth Token.**

Blunk template ID	**Blynk auth token**
Identifies blynk template on cloud	Identifies specific device within template
Allows ESP32 connects to the correct dashboard	ESP32 send data securely to the device dashboard

3. **Why does using DHT22 code with a DHT11 sensor produce incorrect readings? Mention one key difference between the two sensors.**
   Both versions have different timing and resolution. Using DHT22 code with DHT11 sensor can give wrong or false values.
   Key difference is DHT11 (±1°C), 1 Hz sampling. DHT22 (±0.1°C), 0.5 Hz sampling

4. **What are Virtual Pins in Blynk? Why are they preferred over physical GPIO pins for cloud communication?**
   Virtual pins are software pins in Blynk used to send and receive data between ESP32 and cloud. They are preferred over physical pins because no hardware connection needed. It is safe for IOT/cloud communication. It can send any type of data.

5. **What is the purpose of using BlynkTimer instead of delay() in ESP32 IoT applications?**

   Delay blocks the main code and stope the execution of the main program while BlynkTimer is non blocking and it keeps everything running smoothly without blocking other tasks.

Part-B: Long Question

Explain the complete workflow of interfacing ESP32 with Blynk Cloud to display temperature and humidity values.

Your answer should include:

- **Creation of Blynk Template and Datastreams**
  Blynk Template defines the structure of project on cloud and assigns a template id which is used for identification. Datastreams are linked to Virtual Pins to send sensor data to the dashboard. Eg: V0(Temperature), V1(Humidity)
  Blynk template is created by going to developer zone, there create new template, then create dashboard and add new device for it.

- **Role of Template ID, Template Name, and Auth Token**
  Template ID: It identifies blynk template on cloud.
  Template Name: Human-readable name of the template
  Auth Token: Identifies specific device within template

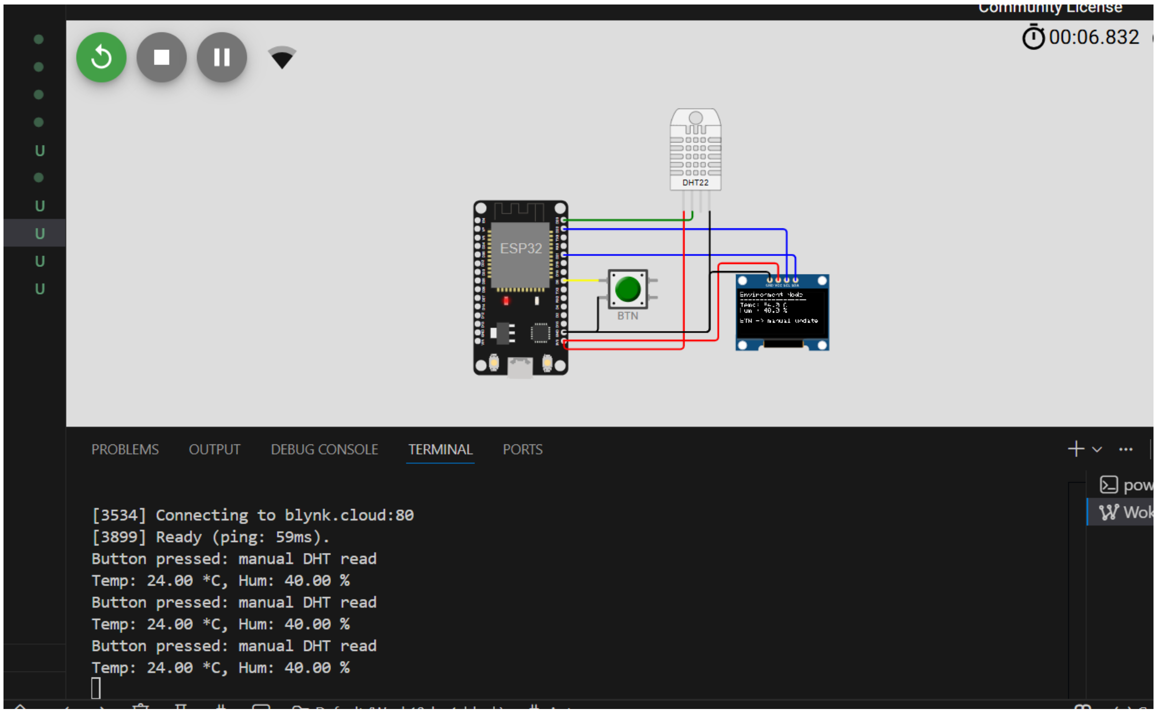
- **Sensor configuration issues (DHT11 vs DHT22)**
Both versions have different timing and resolution. Using DHT22 code with DHT11 sensor can give wrong or false values. Use correct sensor type.
Key difference is DHT11 (±1°C), 1 Hz sampling. DHT22 (±0.1°C), 0.5 Hz sampling

- **Sending data using Blynk.virtualWrite()**
Data read from sensor(temp, humidity) send to the cloud through virtual pins (v0,v1) using Blynk.virtualWrite. These pins are like mailboxes, esp32 drops data into it and Blynk widgets display them.

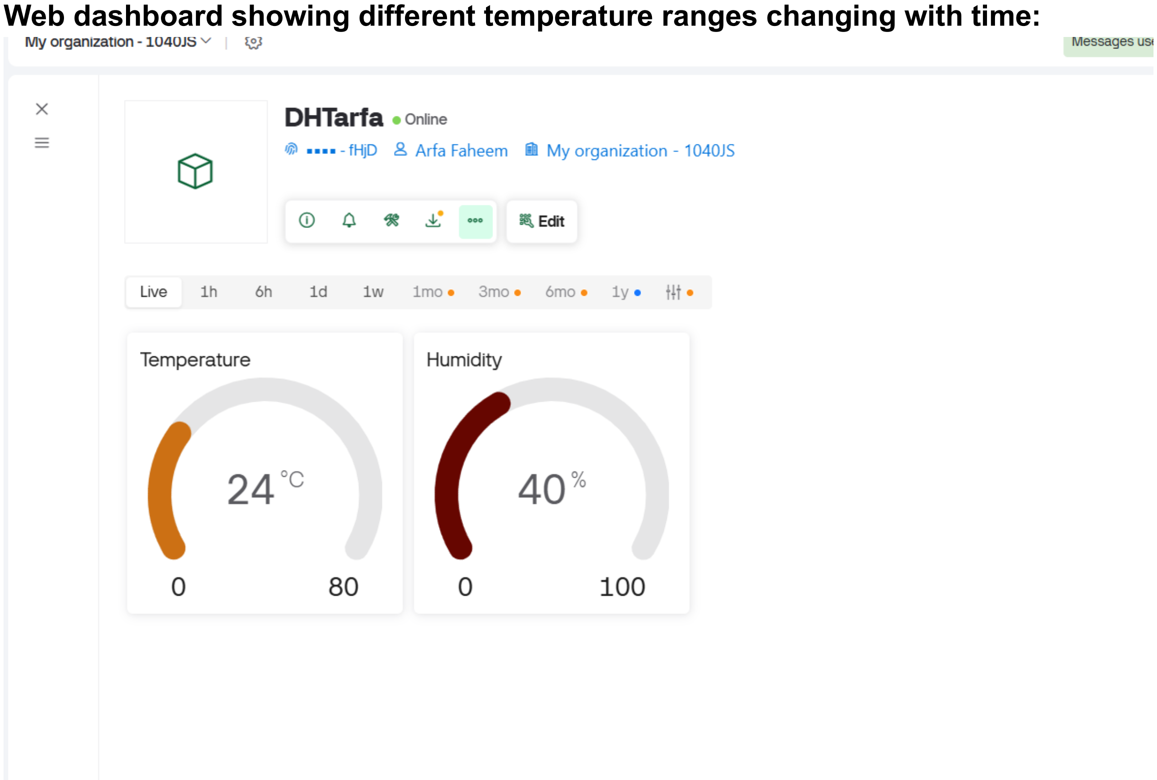
- Common problems faced during configuration and their solutions

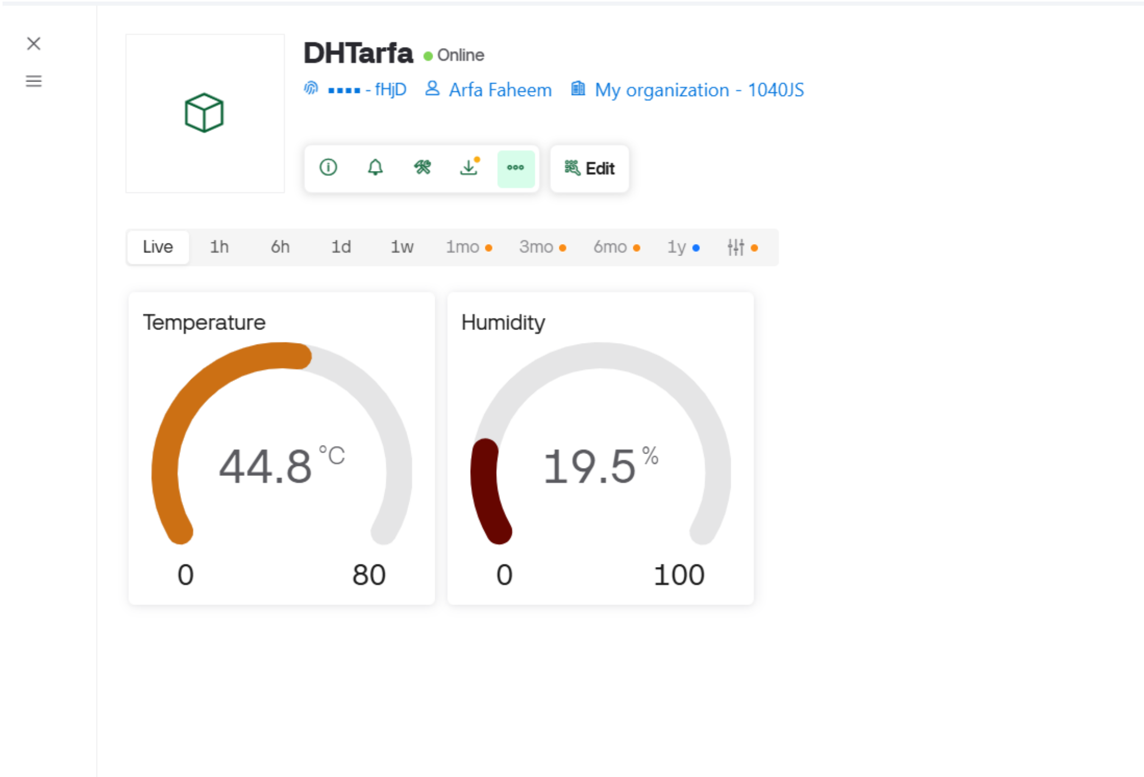
- Wrong readings so should check sensor type is correct or not (dht22 type for dht22 sensor etc)
- Using delay() blocks the other tasks so use BlynkTimer to prevent blocking tasks.
- ESP32 not appearing online, should check the Wi-Fi credentials and auth token.

Screenshot of wokwi simulation:

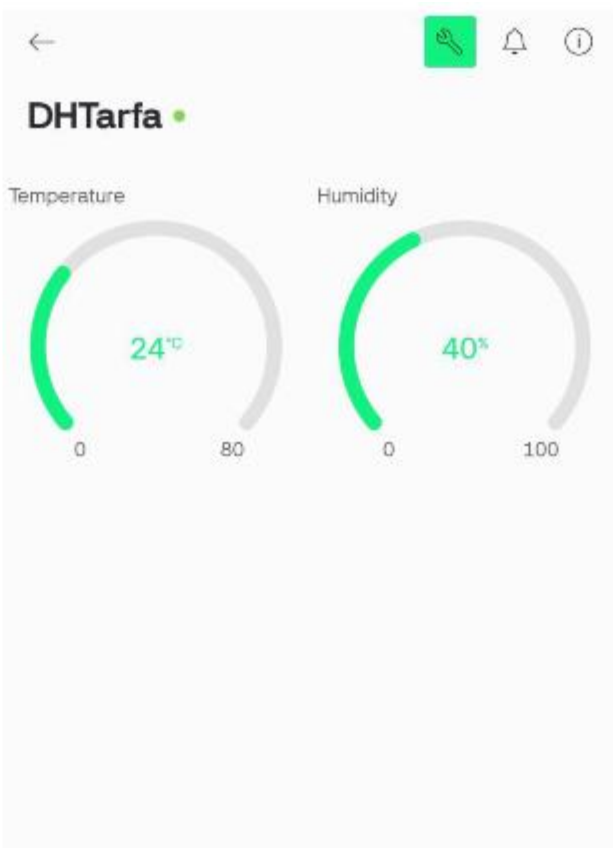


Web dashboard showing different temperature ranges changing with time:





Mobile app dashboard:



Submission Guidelines

- Submit your assignment in PDF format here, and also in the GitHub repository.
- The submission must include a clear description written in your own words.
- Attach screenshots (in pdf file) of the Blynk Cloud (web) and Blynk mobile app dashboards, which you have already created during class.
- Copied content from any source is strictly prohibited. Any student found submitting plagiarized material will be dealt with strictly according to university policy.
- Viva is mandatory and will be conducted for awarding marks.
- The last date of submission is Wednesday, 17th December.